

Explain Snowmen DSA

Production of Snowmen — Simple Explanation

The most important trick in this problem is to **separate the three conveyors into two independent comparisons**.

We need every snowman to satisfy:

$$a < b < c$$

which is equivalent to:

$$a < b \quad \text{and} \quad b < c$$

Let's build the solution from scratch.

1. What is the problem asking?

We have three circular arrays:

```
A = heads
B = torsos
C = legs
```

We choose starting positions:

```
i → starting position in A
j → starting position in B
k → starting position in C
```

Then we create n snowmen:

```
Snowman 1: A[i]    B[j]    C[k]
Snowman 2: A[i+1]  B[j+1]  C[k+1]
Snowman 3: A[i+2]  B[j+2]  C[k+2]
...
```

Because the arrays are circular, indices wrap around.

Every snowman must satisfy:

```
head < torso < legs
```

So for **every position x** :

```
A[i+x] < B[j+x] < C[k+x]
```

2. Key Observation

The condition

```
A[i+x] < B[j+x] < C[k+x]
```

can be split into:

```
A[i+x] < B[j+x]
```

and

```
B[j+x] < C[k+x]
```

Notice something important:

$A \rightarrow B$ depends only on the relative shift between A and B.

Suppose:

```
j = i + d
```

Then we are checking:

```
A[x] < B[x+d]
```

for every x .

Similarly, $B \rightarrow C$ depends only on:

```
k = j + e
```

and we check:

```
B[x] < C[x+e]
```

for every x .

Therefore, we can solve two separate problems:

1. Find how many cyclic shifts of B make $A < B$.
2. Find how many cyclic shifts of C make $B < C$.

Then combine them.

3. Why can we multiply the answers?

Suppose:

```
cntAB = number of valid shifts A → B
cntBC = number of valid shifts B → C
```

For every valid pair of shifts:

```
A → B
B → C
```

we get a valid relative arrangement:

```
A --d--> B --e--> C
```

Now, once these relative shifts are fixed, we can choose the actual starting position i of A in n ways.

For example:

```
i = 0
i = 1
i = 2
...
i = n-1
```

So:

$$\text{answer} = n \times \text{cntAB} \times \text{cntBC}$$

This is the central observation.

4. Building the Intuition

Brute Force

The most obvious solution is:

```
for every i:
  for every j:
    for every k:
      check all n snowmen
```

There are:

$$n^3$$

possible (i, j, k) combinations.

And checking one combination takes $O(n)$.

Therefore:

$$O(n^4)$$

which is far too slow.

5. First Improvement — Remove i

Suppose we fix i and j .

What actually matters?

Only the difference:

$$j - i$$

For example, if:

$$\begin{aligned} i &= 2 \\ j &= 4 \end{aligned}$$

then their relative shift is:

$$d = 2$$

If we instead use:

$$\begin{aligned} i &= 3 \\ j &= 5 \end{aligned}$$

the relative shift is still:

$$d = 2$$

The comparisons are exactly the same, just rotated.

So we don't need to test every (i, j) pair.

There are only n possible cyclic shifts.

6. Check Every Shift

For **A** and **B**, try every shift d :

$$\begin{aligned} d &= 0 \\ d &= 1 \\ d &= 2 \\ &\vdots \\ d &= n-1 \end{aligned}$$

For each shift, check:

$$\begin{aligned} A[0] &< B[d] \\ A[1] &< B[d+1] \\ A[2] &< B[d+2] \\ &\dots \end{aligned}$$

with modulo n .

If **all** comparisons are true, this shift is valid.

That takes:

$$n \text{ shifts} \times n \text{ comparisons}$$

so:

$$O(n^2)$$

Do the same for **B** and **C**.

Total:

$$O(n^2)$$

which is easily fast enough because the sum of n over all test cases is at most **5000**.

7. Example of a Shift

Suppose:

$$\begin{aligned} A &= [1, 2] \\ B &= [3, 4] \end{aligned}$$

Shift $d = 0$

Compare:

```
A[0] < B[0] → 1 < 3 ✓
A[1] < B[1] → 2 < 4 ✓
```

So shift 0 is valid.

Shift $d = 1$

Because the array is circular:

```
B shifted by 1 = [4, 3]
```

Compare:

```
1 < 4 ✓
2 < 3 ✓
```

So shift 1 is also valid.

Therefore:

```
cntAB = 2
```

8. Complete Algorithm

For every test case:

Step 1

Calculate:

```
cntAB = number of shifts d such that
        A[x] < B[(x+d)%n]
        for every x
```

Step 2

Calculate:

```
cntBC = number of shifts d such that
        B[x] < C[(x+d)%n]
        for every x
```

Step 3

Return:

```
answer = n * cntAB * cntBC
```

9. C++ Code

```
C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int countValidShifts(const vector<int>& A, const vector<int>& B) {
    int n = A.size();
    int cnt = 0;

    // Try every cyclic shift of B
    for (int shift = 0; shift < n; shift++) {
        bool valid = true;

        // Check A[x] < B[(x + shift) % n]
        for (int x = 0; x < n; x++) {
            if (A[x] >= B[(x + shift) % n]) {
                valid = false;
                break;
            }
        }

        if (valid) {
            cnt++;
        }
    }

    return cnt;
}
```

```

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;

    while (t--) {
        int n;
        cin >> n;

        vector<int> a(n), b(n), c(n);

        for (int &x : a) cin >> x;
        for (int &x : b) cin >> x;
        for (int &x : c) cin >> x;

        int cntAB = countValidShifts(a, b);
        int cntBC = countValidShifts(b, c);

        long long ans = 1LL * n * cntAB * cntBC;

        cout << ans << '\n';
    }

    return 0;
}

```

10. Understanding the Function

The important function is:

```

C++
int countValidShifts(const vector<int>& A, const vector<int>& B)

```

It answers:

"For how many ways can I rotate B such that every element of A is strictly smaller than the corresponding element of B?"

Try every shift

```

C++
for (int shift = 0; shift < n; shift++)

```

For example:

```

shift = 0
shift = 1
shift = 2
...

```

Compare corresponding elements

```

C++
if (A[x] >= B[(x + shift) % n])

```

We need:

```

A[x] < B[(x + shift) % n]

```

Therefore, if:

```

A[x] >= B[(x + shift) % n]

```

the shift immediately fails.

Why % n?

Because the conveyor is circular.

Suppose:

```

n = 5
x = 4
shift = 2

```

Then:

```

x + shift = 6

```

But index 6 doesn't exist.

Circularly:

```
6 % 5 = 1
```

So we use:

```
C++
B[(x + shift) % n]
```

11. Complete Dry Run — First Sample

Consider:

```
n = 2
A = [1, 2]
B = [3, 4]
C = [5, 4]
```

We calculate:

```
cntAB
cntBC
```

Part 1: $A \rightarrow B$

We need:

```
A[x] < B[x + shift]
```

There are two possible shifts.

Shift = 0

```
A = [1, 2]
B = [3, 4]
```

Check:

```
1 < 3 ✓
2 < 4 ✓
```

Therefore:

```
shift 0 → valid
```

Shift = 1

Circularly, B becomes:

```
[4, 3]
```

Check:

```
1 < 4 ✓
2 < 3 ✓
```

Therefore:

```
shift 1 → valid
```

So:

```
cntAB = 2
```

12. Part 2: $B \rightarrow C$

Now:

```
B = [3, 4]
C = [5, 4]
```

Again, two shifts.

Shift = 0

Compare:

```
3 < 5 ✓
4 < 4 ✗
```

The second comparison fails because the inequality must be **strict**.

Therefore:

```
shift 0 → invalid
```

Shift = 1

Circularly:

```
C = [4, 5]
```

Check:

```
3 < 4 ✓
4 < 5 ✓
```

Therefore:

```
shift 1 → valid
```

So:

```
cntBC = 1
```

13. Combine

We have:

```
n = 2
cntAB = 2
cntBC = 1
```

Therefore:

$$answer = 2 \times 2 \times 1$$

$answer = 4$

which matches the sample.

14. Why the 4 Combinations Exist

We have:

```
2 possible A-B shifts
1 possible B-C shift
2 possible starting positions
```

Therefore:

$$2 \times 1 \times 2 = 4$$

The actual combinations are:

```
(i, j, k)
(1, 1, 2)
(1, 2, 1)
(2, 1, 2)
(2, 2, 1)
```

Exactly as given in the statement.

15. Dry Run of the Second Sample

```
A = [1,1,1]
B = [2,2,2]
C = [3,3,3]
```

Every rotation is identical.

For $A \rightarrow B$:

```
1 < 2
1 < 2
1 < 2
```

So all 3 shifts work:

```
cntAB = 3
```

For $B \rightarrow C$:

```
2 < 3
2 < 3
2 < 3
```

Again:

```
cntBC = 3
```

Therefore:

$$answer = 3 \times 3 \times 3 = 27$$

16. Dry Run of the Third Sample

```
A = [1,2,1,2]
B = [3,3,2,2]
C = [5,5,5,5]
```

The important problem is:

```
A = ...
B = [3,3,2,2]
```

No matter how we rotate B, there will be some position where:

```
A[x] = 2
B[...] = 2
```

Then:

```
2 < 2
```

is false.

So:

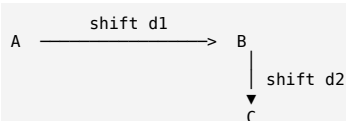
```
cntAB = 0
```

Consequently:

```
answer = n * 0 * cntBC
        = 0
```

17. Mental Model

Think of the problem as connecting three circular conveyors:



We don't care about the absolute starting positions initially.

We only care about:

```
A → B : which rotations work?
B → C : which rotations work?
```

So:

```

valid A→B shifts
  ↓
  ×
valid B→C shifts
  ↓
  ×
n starting positions
  ↓
ANSWER

```

Or mathematically:

$$\text{Answer} = n \times \#(A < B \text{ shifts}) \times \#(B < C \text{ shifts})$$

This is the **main pattern to remember**: when a cyclic construction has conditions like $A < B < C$, check whether the conditions can be split into **independent relative shifts**. Here, that reduces an apparently $O(n^4)$ problem to $O(n^2)$.

Complexity

For `countValidShifts(A, B)`:

- There are n shifts.
- For every shift, we check up to n elements.

Therefore:

$$O(n^2)$$

We do it twice:

$$O(n^2) + O(n^2) = O(n^2)$$

Space

We only store the three arrays:

$$O(n)$$

Final

$$\text{Time: } O(n^2)$$

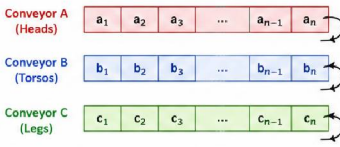
$$\text{Space: } O(n)$$

And because the **sum of n across all test cases is ≤ 5000** , this easily fits within the limits.

PRODUCTION OF SNOWMEN – HOW IT WORKS

1. THE SETUP

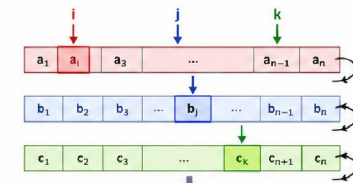
There are 3 conveyors (each of length n).
They are cyclic (circular).
We choose starting positions i, j, k .



After the n -th element, the 1st element comes again.
Example: after a_n comes a_1 .

2. CHOOSING STARTS AND MAKING SNOWMEN

We choose starting indices i, j, k
($1 \leq i, j, k \leq n$).



We build n snowmen as follows:

	Snowman 1	Snowman 2	Snowman 3	...	Snowman n
Head (from A)	a_i	a_{i+1}	a_{i+2}	...	a_{i+n-1}
Torso (from B)	b_j	b_{j+1}	b_{j+2}	...	b_{j+n-1}
Legs (from C)	c_k	c_{k+1}	c_{k+2}	...	c_{k+n-1}

Indices are taken cyclically.
Example: $a_{n+1} = a_1$, $a_{n+2} = a_2$, and so on.

3. CONDITION FOR A STABLE SNOWMAN

For every snowman:



We need this to be true for ALL n snowmen.

Count how many (i, j, k) make all snowmen stable.
That is the answer.

4. HOW THE COUNTING WORKS (KEY IDEA)

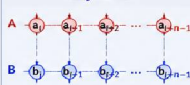
Break the condition:

$$a < b < c$$

is same as

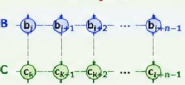
$$a < b \text{ AND } b < c$$

$A < B$ depends only on
the relative shift between A and B.
Let $j = i + d$



We need $a_x < b_{x+d}$ for all x .
There are n possible shifts ($d = 0$ to $n-1$).

$B < C$ depends only on
the relative shift between B and C.
Let $k = j + e$



We need $b_x < c_{x+e}$ for all x .
There are n possible shifts ($e = 0$ to $n-1$).

Combine the counts:

Let
 cntAB = number of shifts d
for which $A < B$ holds for all x
So,
 cntBC = number of shifts e
for which $B < C$ holds for all x

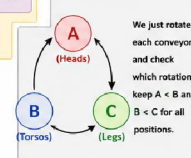
Final Answer

For each valid pair of shifts (d, e) ,
we can choose the starting position
 i of A in n ways (0 to $n-1$).

So,

$$\text{Answer} = n \times \text{cntAB} \times \text{cntBC}$$

Think of it like this:



Yes — “shift” is the most confusing part of this problem. The important thing is:

We are not physically moving the balls.

A shift simply means **choosing a different starting position on the circular conveyor.**

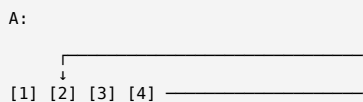
Let's understand this very carefully.

1. First forget the word "shift"

Suppose:

$A = [1, 2, 3, 4]$
 $B = [5, 6, 7, 8]$
 $C = [9, 10, 11, 12]$

The conveyors are circular:



After 4, we come back to 1.

So:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$

Same for B and C.

2. What does choosing i, j, k mean?

Suppose we choose:

$i = 1$
 $j = 1$
 $k = 1$

Then we start all three conveyors from their first positions:

A: 1 2 3 4
↑
B: 5 6 7 8
↑
C: 9 10 11 12

↑

We create:

```
Snowman 1: (1, 5, 9)
Snowman 2: (2, 6, 10)
Snowman 3: (3, 7, 11)
Snowman 4: (4, 8, 12)
```

3. Now change only j

Suppose:

```
i = 1
j = 2
k = 1
```

We are now starting the B conveyor from 2 instead of 1.

So:

```
A: [1] [2] [3] [4]
    ↑
B: [5] [6] [7] [8]
    ↑
C: [9] [10] [11] [12]
    ↑
```

Now the snowmen become:

```
Snowman 1 → (1, 6, 9)
Snowman 2 → (2, 7, 10)
Snowman 3 → (3, 8, 11)
Snowman 4 → (4, 5, 12)
```

This is what we call a shift.

B is effectively **one position ahead** of A.

So:

```
j - i = 1
```

Therefore:

```
shift = 1
```

4. Visualize the shift

Take:

```
A = [1, 2, 3, 4]
B = [5, 6, 7, 8]
```

Shift = 0

Corresponding elements:

```
A:  1  2  3  4
   ↓  ↓  ↓  ↓
B:  5  6  7  8
```

So:

```
1 ↔ 5
2 ↔ 6
3 ↔ 7
4 ↔ 8
```

Shift = 1

Now B starts one position later:

```
A:  1  2  3  4
   ↓  ↓  ↓  ↓
B:  6  7  8  5
```

Because B is circular.

So:

```
1 ↔ 6
2 ↔ 7
3 ↔ 8
4 ↔ 5
```

Shift = 2

```
A:  1  2  3  4
   ↓  ↓  ↓  ↓
B:  7  8  5  6
```

So:

```
1 ↔ 7
2 ↔ 8
3 ↔ 5
4 ↔ 6
```

Shift = 3

```
A:  1  2  3  4
   ↓  ↓  ↓  ↓
B:  8  5  6  7
```

So:

```
1 ↔ 8
2 ↔ 5
3 ↔ 6
4 ↔ 7
```

5. Why are there exactly n shifts?

Because the conveyor has n positions.

For:

```
n = 4
```

we have:

```
shift = 0
shift = 1
shift = 2
shift = 3
```

After shift 4, we are back to shift 0.

```
shift 0
  ↓
shift 1
  ↓
shift 2
  ↓
shift 3
  ↓
shift 4 = shift 0
```

That's because the conveyor is circular.

6. Now connect this to the actual problem

This is the key part.

Suppose:

```
A = [1, 2]
B = [3, 4]
```

We want:

```
A < B
```

for every snowman.

Shift 0

```
A: 1  2
   ↓  ↓
B: 3  4
```

Check:

```
1 < 3 ✓
2 < 4 ✓
```

Therefore:

```
shift 0 = VALID
```

Shift 1

Because B is circular:

```
B = [3, 4]
shift by 1
     ↓
B = [4, 3]
```

Now:

```
A: 1  2
   ↓  ↓
B: 4  3
```

Check:

```
1 < 4 ✓
2 < 3 ✓
```

Therefore:

```
shift 1 = VALID
```

So:

```
cntAB = 2
```

7. But where does this shift come from in i and j?

This is **very important**.

Suppose:

```
i = 1
j = 2
```

Then:

```
j - i = 1
```

So B is shifted by 1 relative to A.

Therefore:

```
shift = (j - i + n) % n
```

For example:

```
i = 1
j = 2
n = 4
shift = 2 - 1
       = 1
```

Another example:

```
i = 3
j = 1
n = 4
```

Then:

```
j - i = 1 - 3 = -2
```

But because the conveyor is circular:

```
shift = (-2 + 4) % 4
       = 2
```

So the relative shift is 2.

8. The BIG insight

The original problem asks us to choose:

```
i, j, k
```

That's apparently:

```
n × n × n = n3
```

possibilities.

But we realize:

A and B only care about their relative position.

Instead of caring about:

```
i = 1, j = 2
i = 2, j = 3
i = 3, j = 4
...
```

all of these have the same relative shift:

```
j - i = 1
```

For example:

```
i=1, j=2    → shift 1
i=2, j=3    → shift 1
i=3, j=4    → shift 1
i=4, j=1    → shift 1 (circularly)
```

They all produce the **same relative alignment**.

That's why we count shifts instead of every (i,j) pair.

9. Same idea for B → C

Exactly the same thing happens between B and C.

We define:

```
shiftBC = k - j
```

So:

```
A ---- shiftAB ----> B ---- shiftBC ----> C
```

For example:

```

A -----> B
      shiftAB = 1
      |
      | shiftBC = 2
      v
      C

```

We independently check:

```
A[x] < B[x + shiftAB]
```

and:

```
B[x] < C[x + shiftBC]
```

10. Why the answer becomes $n \times \text{cntAB} \times \text{cntBC}$

Imagine we found:

```
cntAB = 2
cntBC = 3
```

So there are:

```
2 choices for A → B alignment
3 choices for B → C alignment
```

Therefore:

```
2 × 3 = 6
```

different relative arrangements.

But we haven't selected the **absolute starting point** yet.

We can put the whole arrangement starting from any position in A:

```
i = 0
i = 1
i = 2
...
i = n-1
```

That's n possibilities.

Therefore:

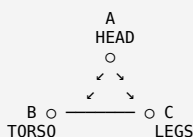
```
ANSWER
├── n choices of starting position
├── cntAB valid shifts
└── cntBC valid shifts
```

Hence:

$$\text{answer} = n \times \text{cntAB} \times \text{cntBC}$$

11. The easiest mental model

Think of **three rotating rings**:



We don't physically move the balls.

We simply ask:

If I start conveyor B at a different position relative to A, does every A ball remain smaller than its corresponding B ball?

That's an $A \rightarrow B$ shift.

Then:

If I start conveyor C at a different position relative to B, does every B ball remain smaller than its corresponding C ball?

That's a $B \rightarrow C$ shift.

So **shift** = relative starting position between two circular conveyors.

One sentence to remember

In this problem, a shift means: "How many positions ahead/behind is the starting point of one circular array compared with another array?"

And in code:

```
C++  
B[(x + shift) % n]
```

means:

"Take the element of B that is **shift** positions away from position x, wrapping around because B is circular."

That single line is essentially the whole meaning of **shift** in this problem.

ChatGPT can make mistakes. Check important info.